Apache Hadoop in Distributed Software Systems: A Comparative Analysis Against Apache Spark for Big Data Processing

Mohamed Alif Fathi bin Abdul Latif
Faculty of Computing
Universiti Teknologi Malaysia
Johor Bahru, Malaysia
mohamedaliffathi@graduate.utm.my

***Abstract*— This paper discusses Apache Hadoop as a system used to manage and process large amounts of data. It explains its main components, how it works and where it is used. The paper also compares Apache Hadoop with Apache Spark in terms of performance, speed, memory usage and other features. From the comparison, Apache Spark is faster for tasks that need repeated processing and real-time data, while Apache Hadoop is still useful for batch processing. Overall, the choice between Apache Hadoop and Apache Spark depends on the type of task and system resources.**

Keywords—Apache Hadoop, distributed computing, MapReduce, big data, Apache Spark, HDFS, performance benchmarking

I. Introduction

In todays era, the amount of data produced has increased significantly. Many companies such as healthcare, finance and e-commerce producing data continuously at very large scale. Because of this, traditional data management like relational databases cannot handle it properly. Due to these limitation, distributed computing have become more important. These system can allow data to be processed across multiple machines at the same time. One of the most widely used is Apache Hadoop. It can provide a way to store and process big data using clusters, making it more scalable and cost-effective (Azeroual & Fabre, 2021). Apache Hadoop has been adopted by many organizations. The core components of Apache Hadoop, Hadoop Distributed File System (HDFS) and MapReduce are the foundation of data processing capabilities. However, Apache Hadoop limitation which is the disk-based processing model become more problematic as more organization needs faster processing, real-time analytics and machine learning computing tasks. Because of that, Apache Spark was introduced to address the problem. It have in-memory processing, which is significantly improve the performance. But, Apache Hadoop still relevant to this days for batch processing and data storage. This paper aims to analyse Apache Hadoop in term of its architecture and features and also make a comparative analysis with Apache Spark for better understanding their differences and suitable use cases.

II. Architecture and Core Components

A. Overview

Apache Hadoop is an open-source distributed software that stored large amount of dataset and processed using clusters of machine. One of the advantages of using Apache Hadoop is it can scale horizontally which means it can add new machines rather than just upgrade current machine.

B. Hadoop Distributed File System (HDFS)

HDFS is the storage management for Apache Hadoop. The way of the data stored in HDFS by splitting large files into fixed-sized smaller block and distributing them across different nodes in a cluster. There are two main types of nodes, which are NameNode and DataNodes. NameNode manage the metadata and control the system while DataNodes store the actual data. HDFS also ensure fault tolerance by replicated each block across multiple nodes but this will increase the storage usage.

C. MapReduce

MapReduce is the processing model that used in Apache Hadoop. It divide tasks into two main stages which are Map and Reduce. The Map stage processes input and generates key-value pairs, while the Reduce stage combine these two into final output. One of the MapReduce characteristic is that the data is written to disk between stages. This will increase reliability but it also created input/output (I/O) overhead. This become a drawback for task that required multiple iterations like machine learning. Because of this, Apache Hadoop perform well in batch processing but less efficient for workload that needed repetitive processing. Fig 1 below shows the architecture of MapReduce.

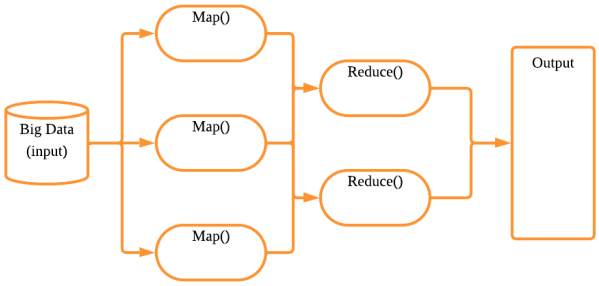

Fig 1. The MapReduce process in Apache Hadoop (inspired by Azhir et al., 2022)

III. Real-World Case Study

A real-world example of Hadoop can be seen during the COVID-19 pandemic. It was used by public health organizations and researchers to process large datasets from multiple sources, including patient records (Azeroual and Fabre, 2021). HDFS provided the storage needed to manage these large datasets, while MapReduce more compatible for large-scale analysis. This allowed researchers to identify patterns such as infection rates and population risk levels. Although Apache Hadoop is not suitable for real-time processing, it is very effective for large-scale batch analysis and long-term data processing tasks.

IV. Comparative Analysis

Apache Spark was developed at UC Berkeley's AMPLab as a response to the limitations of Apache Hadoop's disk-based MapReduce model (Ahmed et al., 2020). While both

frameworks are designed for distributed data processing, they have different processing models, performance characteristics and use case suitability which is will be discussed the differences in this section. Table I provides a summary of the comparison.

TABLE I. COMPARATIVE ANALYSIS OF APACHE HADOOP AND APACHE SPARK

Criteria	Apache Hadoop (MapReduce)	Apache Spark
Processing Model	Disk-based batch processing (Ahmed et al., 2020)	In-memory processing (Ahmed et al., 2020)
Speed	Slower due to high disk I/O overhead (Mostafaeipour et al., 2021)	1.4 times faster than Apache Hadoop on machine learning workloads due to in-memory caching (Mostafaeipour et al., 2021)
Fault Tolerence	Data replication across HDFS nodes (Filaly et al., 2025)	RDD lineage graph-based recovery. It Recomputes lost partitions without full data replication (Ahmed et al., 2020)
Memory Usage	Low memory requirement. It relies on disk storage making it suitable for memory-constrained environments (Sewal & Singh, 2024)	Higher RAM requirement. Performance degrades when dataset exceeds available memory, causing disk spill (Sewal & Singh, 2024)
Streaming Support	No native real-time streaming. It is batch-oriented by design (Fernandez-Gomez et al., 2023)	Supports micro-batch and structured streaming using Spark Streaming. It is suitable for real-time analytics (Fernandez-Gomez et al., 2023)
Best Use Case	Large-scale batch processing, ETL pipelines, simple one-pass data transformations on commodity clusters (Azeroual & Fabre, 2021)	Iterative machine learning, real-time streaming analytics, interactive queries, and graph processing (Fernandez-Gomez et al., 2023)

A. Processing Model and Speed

The main difference between Apache Hadoop and Apache Spark is how they process data. MapReduce in Apache Hadoop saves all the intermediate results to disk after each stage, which causes a lot of I/O overhead. Apache Spark processes data in memory using Resilient Distributed Dataset (RDD) and also uses a Directed Acyclic Graph (DAG) scheduler. Because of this, Apache Spark can reduces the usage of iteration for disk read and write operations.

Mostafaeipour et al. (2021) tested the performance by implementing the K-Nearest Neighbor (KNN) algorithm on datasets with different sizes in both frameworks. The result shows that Apache Spark have 40 percent advantages in term of processing speed over the Apache Hadoop for all dataset sizes tested. Besides, Apache Hadoop was found to use more CPU and network resources compared to Spark for the same tasks.

Ahmed et al. (2020) also support this finding using a larger dataset of 600 GB with the HiBench benchmarking tool. The result shows that Apache Spark performs better for iterative tasks, because it cache data in memory and reuse it. However, Sewal and Singh (2024) found some interesting point. They tested both systems using gradient-boosted tree regression with HIGGS and COVID-19 datasets. They found that Apache Spark is faster only when the cluster has enough memory to store the whole dataset. If the dataset is bigger than the available memory, Apache Spark has to write data to disk making their performance drop. In this scenario, Apache Hadoop can perform better eventhough using disk. This is important for organization because system performance not only depends on speed but also hardware resources.

B. Streaming and Real-Time Analytics

One of the limitations of Apache Hadoop is that it does not support real-time data streaming. Apache Hadoop is designed for batch processing. The data needs to be collected first then stored in HDFS, then processed in batches.

Fernandez-Gomez et al. (2023) developed a framework using Apache Spark for big data streaming in IoT networks. The results shows Spark Streaming, which uses a micro-batch processing model. It support real-time analytics with very low latency. They applied this framework to time-series forecasting using sensor data streams. This technique is something that Hadoop MapReduce cannot do.

For organizations that need continuous data processing and real-time decision making like network monitoring, Apache Spark is more suitable compared to Apache Hadoop because of its streaming capability.

V. CONCLUSION

In conclusion, Apache Hadoop remains an important framework for big data analytics even with newer technologies like Apache Spark. Apache Spark offers significant performance advantages which is better for while Hadoop is more suitable for batch processing and environments with limited memory. Additionally, Hadoop provides a more mature security framework, making it suitable for sensitive and regulated data environments. Rather than replacing Apache Hadoop, Apache Spark is often used alongside it in modern data architectures. Future research can focus on improving the integration of both frameworks and addressing challenges related to performance optimisation and data security.

REFERENCES

[1] Ahmed, N., Barczak, A. L. C., Susnjak, T., & Rashid, M. A. (2020). A comprehensive performance analysis of Apache Hadoop and Apache Spark for large scale data sets using HiBench. *Journal of Big Data, 7*, Article 110. https://doi.org/10.1186/s40537-020-00388-5

[2] Azeroual, O., & Fabre, R. (2021). Processing big data with Apache Hadoop in the current challenging era of COVID-19. *Big Data and Cognitive Computing, 5* (1), 12. https://doi.org/10.3390/bdcc5010012

[3] Azhir, E., Hosseinzadeh, M., Khan, F., & Mosavi, A. (2022). Performance evaluation of query plan recommendation with Apache Hadoop and Apache Spark. *Mathematics, 10*(19), 3517. https://doi.org/10.3390/math10193517

[4] Fernández-Gómez, A. M., Gutiérrez-Avilés, D., Troncoso, A., & Martínez-Álvarez, F. (2023). A new Apache Spark-based framework for big data streaming forecasting in IoT networks. *The Journal of Supercomputing, 79*(10), 11078–11100. https://doi.org/10.1007/s11227-023-05100-x

[5] Filaly, Y., Mohammed, A. S., Elidrissi, A. G., & Ghzala, R. B. (2025). A comprehensive survey on big data privacy and Hadoop security: Insights into encryption mechanisms and emerging trends. *Results in Engineering, 27*, Article 106203. https://doi.org/10.1016/j.rineng.2025.106203

[6] Mostafaeipour, A., Rafsanjani, A. J., Ahmadi, M., & Dhanraj, J. A. (2021). Investigating the performance of Hadoop and Spark platforms

on machine learning algorithms. *The Journal of Supercomputing, 77*(2), 1273–1300. https://doi.org/10.1007/s11227-020-03328-5

[7] Sewal, P., & Singh, H. (2024). Performance comparison of Apache Spark and Hadoop for machine learning based iterative GBTR on HIGGS and Covid-19 datasets. *Scalable Computing: Practice and Experience, 25*(3). https://doi.org/10.12694/scpe.v25i3.2687